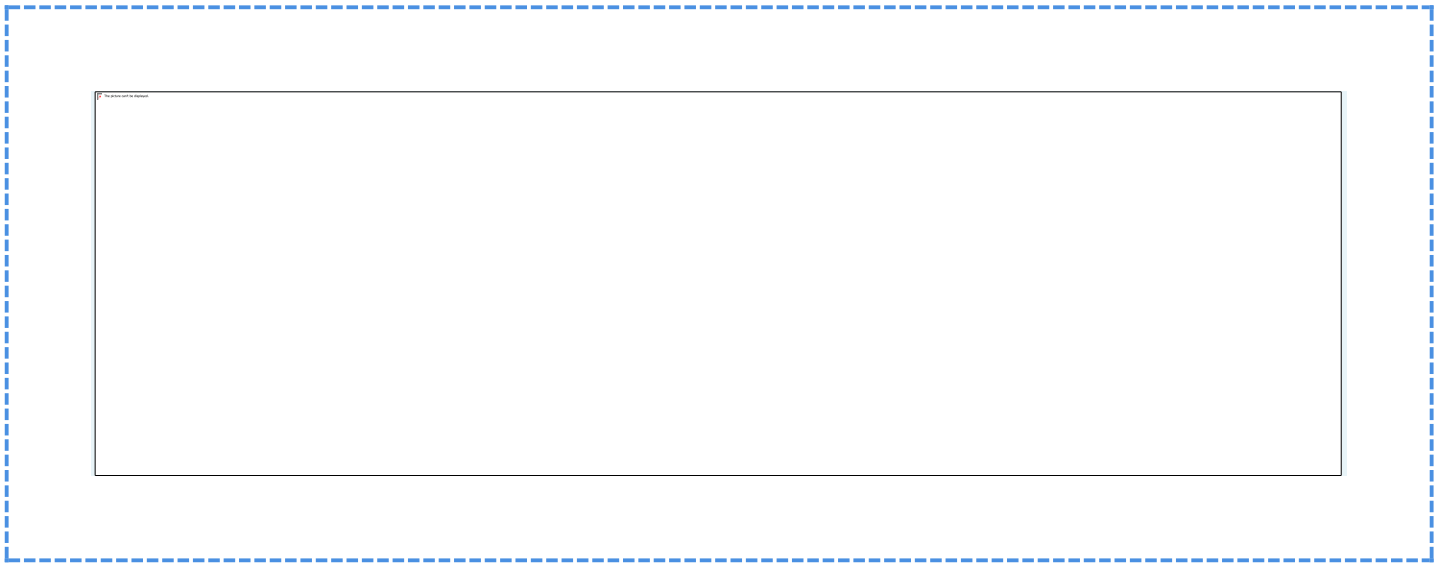
Foxhound Unit

Communication Mechanism & Data Flow



1. Purpose

This project implements a distributed solar farm monitoring system that simulates, ingests, processes, stores, and visualizes real-time telemetry from large fleets of solar panels. The system is designed to operate under realistic conditions such as intermittent connectivity, high message volume, and hardware or communication faults.

The project is developed in two phases:

- **Phase 1:** Design and implementation of a distributed ingestion pipeline
- **Phase 2:** Development of a remote dashboard for visualization and monitoring

2. Components (Files & Purpose)

- `src/solar_panel_telemetry.py` — Telemetry generator (CSV or JSONL)
- `src/edge_collector_server.py` — TCP collector; receives lines, validates them, batches, compresses, and publishes
- `src/central_ingest.py` — MQTT subscriber; deduplicates and writes to a datastore or fallback JSONL
- `src/records_handler.py` — Validation/normalization utilities that ensure the correct 13-field schema

- `src/ingest_fallback.jsonl` — Fallback file for ingest when InfluxDB is not configured
- `web/influx_api.py` — REST API that serves aggregated and panel-level telemetry to the dashboard.
- `web/influx_writer.py` — Generates and writes sample telemetry data into InfluxDB for testing.
- `web/dashboard.html` — Web dashboard that visualizes live system metrics, time-series data, and faults.
- `data/` — Sample files
- `tests/` — Pytest tests validating the flow and components (test_phase1.py)

3. Chosen Communication Mechanism — Rationale

Generator → Collector

- **Transport:** TCP (plain text / line-delimited JSONL or CSV)
- **Rationale:** Simplicity and compatibility. The generator can pipe output using netcat (nc) or simple TCP clients to the collector's listening port. This also allows running the generator and collector locally without an MQTT broker.

Collector → Cloud/Backend

- **Transport:** MQTT (Paho MQTT client is used)
- **Rationale:** MQTT provides durable, decoupled pub/sub between the edge and central services. The collector can batch and publish telemetry messages to a topic while the central ingest remains subscribed. MQTT is lightweight and suits intermittent connectivity.

Persistence / Buffering

- SQLite is used as an outbox on the edge (`edge_buffer.db`) to queue compressed publish payloads when MQTT is unavailable. This ensures messages are not lost when network connectivity is intermittent.
- Central ingest uses a deduplication SQLite DB (`ingest_dedup.db`) to prevent duplicate insertions.

Serialization & Compression

- JSONL is the primary message format in the pipeline. The collector compresses batches using gzip before publishing to MQTT to conserve bandwidth.

4. Data Schema & Validation

Each telemetry record follows the final schema of 13 fields:

`timestamp_utc, panel_id, string_id, status, fault, power_w, voltage_v, current_a, irradiance_wm2, ambient_temp_c, cell_temp_c, orientation_deg, tilt_deg`

- **Validation:** The collector uses RecordsHandler (`src/records_handler.py`) to reject lines that do not match the expected schema. CSV input will be converted (header must match required fields).
- **Deduplication:** Central ingest deduplicates messages using the composite of timestamp and panel_id (tests reflect dedup logic). Duplicate messages are dropped.

5. Data Flow (Step-by-Step)

1. Telemetry Generation

The generator (`src/solar_panel_telemetry.py`) produces CSV lines or JSONL lines for configured panels and time windows.

Example:

```
python src/solar_panel_telemetry.py --panels 10 --hours 1 --step 30 --format jsonl
```

2. Push to Collector

The generator streams lines to the collector TCP endpoint, e.g., `| nc localhost 9000`.

Collector receives and buffers incomplete chunks until newline-terminated records are ready. The collector handles message chunking and line re-assembly.

3. Validation & Batching

For each line, RecordsHandler validates the schema. Valid records are appended to the current batch; invalid records are counted and discarded.

The collector batches by count (configurable batch_size) or by time (batch_secs). When limit reached, it compresses the batch (gzip) and publishes via MQTT.

4. Publish to MQTT

The collector publishes to a topic `panels/{site_id}/telemetry`. The payload is the gzip-compressed JSONL string representing the batch.

Example: `panels/site-01/telemetry`

If MQTT is down or unavailable, the collector persists the compressed batch in the SQLite outbox (`edge_buffer.db`).

5. MQTT Ingest

Central ingest subscribes to the topic(s) and receives compressed batches. It decompresses and parses each JSON line.

For each record the central ingest:

- Deduplicates using `ingest_dedup.db` (if message identical by panel_id + timestamp, skip)
- Writes to InfluxDB if configured (by environment variables). If Influx is not available/configured, it writes to `src/ingest_fallback.jsonl`

6. Acknowledgment & Retry

The collector sets stats for messages published and queued.

A background loop sends outbox items when connectivity is restored, ensuring eventual delivery.

6. Failure Modes & Handling

Failure Mode	Handling Strategy
Collector MQTT disconnect	Batches are written to the local SQLite outbox (edge_buffer.db). The send_outbox_loop attempts to retransmit outbox entries once connection is restored.
Invalid records	Rejected and tracked (stats["messages_invalid"] incremented)
Duplicate records on central ingest	Central ingest drops duplicates based on panel_id + timestamp; duplicates are not forwarded to datastore
Parsing and decompression failures	Caught by central ingest and can fallback to ingest_fallback.jsonl

7. Test Summary (Verification)

Tests validate RecordsHandler, collector logic (line handling and batching), outbox flow (push/pop and retries), and central ingest dedup behavior.

Running Tests

Prepare environment & dependencies:

```
python -m venv .venv

./.venv/Scripts/Activate.ps1   # Windows PowerShell

pip install -r requirements.txt

pip install pytest
```

Execute tests:

```
pytest -q
```

Results from the Test Run

15 tests passed, 2 warnings

8. How to Run the Minimal Pipeline Locally

1. (Optional) Start an MQTT broker (mosquitto) locally:

```
# e.g. on macOS / Linux:
```

```
mosquitto
```

2. Start central ingest (subscribe to MQTT):

```
python src/central_ingest.py --mqtt-host localhost --mqtt-port 1883
```

3. Start edge collector:

```
python src/edge_collector_server.py --mqtt-host localhost --mqtt-port 1883 --site-id
site-01 --port 9000
```

4. Generate and send telemetry:

```
python src/solar_panel_telemetry.py --panels 5 --hours 0.1 --format jsonl | nc
localhost 9000
```

9. Configuration Notes

- Central ingest can write to InfluxDB; configure environment variables `INFLUX_URL`, `INFLUX_TOKEN`, `INFLUX_ORG`, `INFLUX_BUCKET` to enable.
- Default fallback sink if Influx not configured: `src/ingest_fallback.jsonl`
- Default outbox DB path on the collector: `edge_buffer.db`. Default dedup DB path on ingest: `ingest_dedup.db`. Change these in the module or via environment/config as desired.

10. Data Security & Privacy

- Telemetry data used here is synthetic and for demonstration/testing.
- Do not commit secrets; use environment variables to pass credentials for production services like InfluxDB.

11. Limitations & Observations

- Telemetry deduplication is based on panel_id + timestamp only (adjust if more robust checks needed).
- Exact message ordering and concurrency interleaving varies — tests are deterministic where necessary by the test harness and the generator can be seeded for deterministic outputs if needed.
- For guaranteed exactly-once semantics across unreliable networks, further measures (transactional storage and idempotent insertion) would be needed.

12. Appendix: Useful Commands

Run unit tests:

```
pytest
```

Generate CSV:

```
python src/solar_panel_telemetry.py --panels 100 --hours 1 --step 60 --out
telemetry.csv
```

Start collector listening on port 9000 and publish to MQTT:

```
python src/edge_collector_server.py --mqtt-host localhost --mqtt-port 1883 --site-id
site-01 --port 9000
```

Run central ingest:

```
python src/central_ingest.py --mqtt-host localhost --mqtt-port 1883
```

Transition from Phase 1 to Phase 2

Phase 1 focused on reliable ingestion and transport of telemetry data from solar panel emulators to a centralized storage system. While Phase 1 ensured scalability and fault tolerance, it did not provide direct visibility into system behavior.

Phase 2 builds directly on Phase 1 by adding a visualization and analytics layer that consumes the processed telemetry and presents it to administrators in real time. No changes were made to the ingestion pipeline, instead, Phase 2 extends the system by integrating a backend API and a web-based dashboard.

1. Phase 2 – Remote Dashboard & Data Visualization

1.1 Phase 2 Objective

The objective of Phase 2 is to design and implement an interactive remote dashboard capable of visualizing real-time and historical telemetry data received from the Phase 1 pipeline. The dashboard enables system operators to monitor solar farm performance, detect faults, and analyze environmental conditions.

1.2 Backend Integration

A Flask-based REST API was implemented to serve processed telemetry from InfluxDB to the dashboard. The API performs aggregation and filtering to reduce frontend complexity.

Key API Endpoints:

Endpoint	Purpose
/api/current	Latest telemetry per panel
/api/aggregate	System-level aggregated metrics
/api/faults	Active faults and warnings
/api/panel/<id>	Historical panel-level data

To accurately represent real-time activity, panels are classified as active only if their most recent timestamp falls within a fresh window, ensuring that panels which stop sending data are correctly marked as inactive.

1.3 Dashboard Implementation

The remote dashboard is implemented as a single-page web application using standard web technologies:

- HTML5 and CSS3
- JavaScript
- Chart.js for time-series visualization

The dashboard layout is designed for clarity, with clear labels, visual grouping, and color-coded indicators.

1.4 Visualized Metrics and Analytics

The dashboard provides real-time visibility into both system-level and panel-level metrics.

System-Level Metrics:

- Total generated power (kW)
- Number of active panels
- Average irradiance (W/m²)
- Active fault count

Time-Series Visualizations:

- Total system power output over time
- Irradiance and ambient temperature trends
- Panel-level historical performance

Fault & Alert Indicators:

- Active warnings and faults
- Severity-based color coding
- Timestamped event reporting

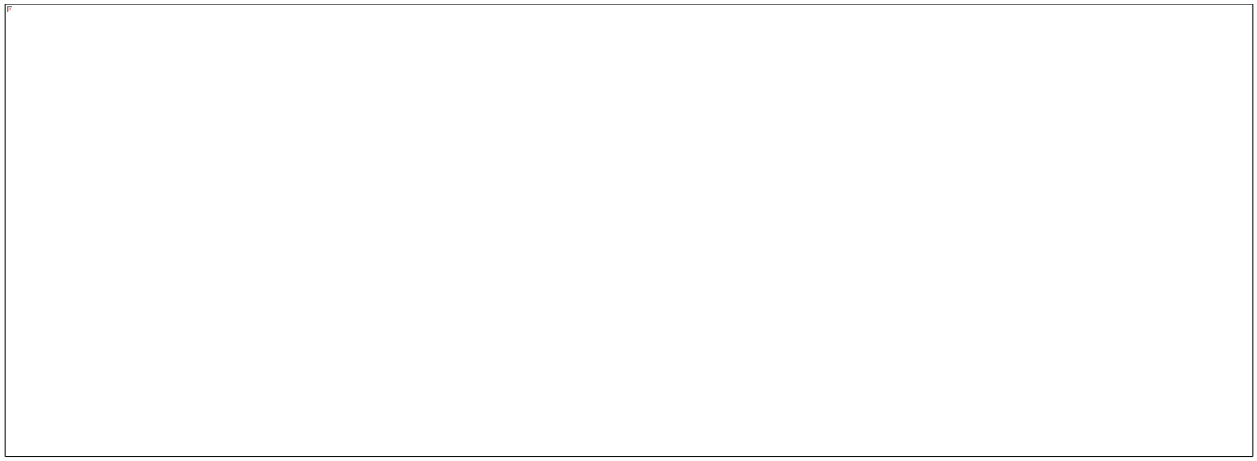
These visualizations allow administrators to quickly assess system health and performance.

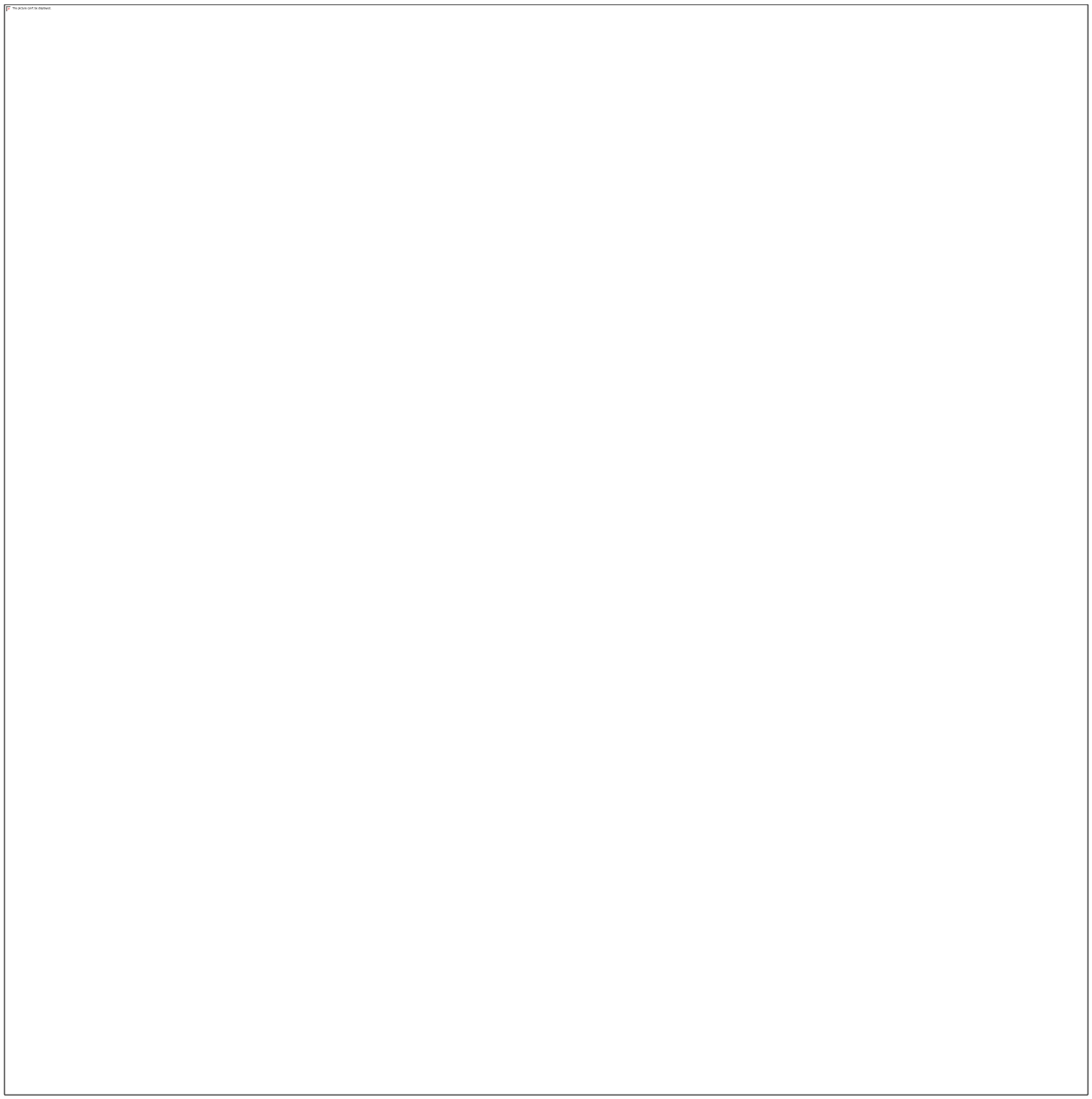
1.5 Panel and String Status Overview

Each solar panel is represented visually with a status indicator:

- **Green:** Normal operation
- **Yellow:** Warning (e.g., soiling)
- **Red:** Fault (e.g., string open)

String-level aggregation enables monitoring of subsystem performance and helps identify localized issues.





2. Phase 2 Demonstration & Testing

2.1 Real-Time Data Flow

Telemetry generated by the emulators flows through the Phase 1 pipeline and appears on the dashboard within seconds. Changes in power, irradiance, temperature, and faults are reflected in real time.

2.2 Fault Simulation

Multiple fault scenarios were tested, including:

- String open circuit faults
- Panel soiling warnings
- Thermal hotspots
- Shading conditions

Each fault produced realistic effects on system output and was correctly visualized by the dashboard.

2.3 Load and Stability Testing

- Multiple panels were simulated concurrently
- Backend API sustained continuous requests
- Dashboard remained responsive under load
- No data loss or inconsistencies were observed

Conclusion

The completed system integrates a robust distributed ingestion pipeline (Phase 1) with a fully functional remote monitoring dashboard (Phase 2). The final solution provides real-time visibility, historical analysis, and fault monitoring, closely resembling industrial solar farm monitoring platforms.